

Provably Complete Generalized Planning with LLMs

Katharina Stein¹, Chaahat Jain¹, Jörg Hoffmann^{1,2}, Alexander Koller¹

¹Saarland Informatics Campus, Saarland University, Saarbrücken, Germany

²German Research Center for Artificial Intelligence (DFKI), Saarbrücken, Germany

Correspondence: kstein@lst.uni-saarland.de

Abstract

Generalized planning aims to compute a plan that solves all instances of a planning domain. Recent work has used LLMs to automatically generate and debug such generalized plans in the form of Python programs and achieved perfect test data coverage for several domains. However, whether these generalized plans are actually complete, i.e. solve all instances of the domain, could only be determined by manual evaluation. Here, we present an approach for automatically generating generalized plans in Lean together with proofs of their completeness relative to the specification of the domain constraints provided as input. We introduce a semantic-preserving PDDL-to-Lean conversion, and use an LLM to generate both the generalized plan and the formal proof that it solves every instance satisfying the domain constraints. The correctness of the completeness proof is determined by Lean’s kernel. We evaluate our approach on 13 domains, using GPT-5.6-Sol as the LLM. For 12 of the domains we obtain generalized plans together with valid proofs that the generalized plans solve all problem instances satisfying the domain constraints. For the remaining domain, we obtain a generalized plan with perfect coverage on the test set.

Introduction

Generalized planning aims to compute a plan that solve not only a single planning task, but all tasks from a given PDDL domain. Such a strategy will typically contain loops and conditionals to deal with different cases and sizes of tasks. For example, in the Spanner domain from the IPC Learning track 2023 a man can walk between linked locations, pick up spanners and use a spanner to tighten a loose nut which makes the spanner unusable (Taitler et al. 2024). As per the IPC problem generator, the links between locations form a one-way path from the initial location (a shed) to the location of all loose nuts (a gate), and all usable spanners are located between the shed and the gate. The goal is to tighten all loose nuts. Figure 1 shows an example planning task and a strategy that generalizes to all tasks of the domain.

A variety of approaches for formalizing and obtaining generalized plans have been proposed, including representing generalized plans as synthesized programs with loops and conditionals (e.g. Srivastava, Immerman, and Zilberstein 2011; Jiménez, Segovia-Aguas, and Jonsson 2019) or general policies (e.g. Bonet, Francès, and Geffner 2019; Bonet and Geffner 2018). Recently, there has been work on using Large



Move from the shed to the gate. At each intermediate location, pick-up all spanners. When reaching the gate, tighten each loose nut using a different spanner.

Figure 1: Illustration of an example planning task from the Spanner domain and a strategy for solving all Spanner tasks.

Language Models (LLMs) to generate, and debug, generalized plans in the form of Python programs (Silver et al. 2024; Stein et al. 2026), with Stein et al. (2026) achieving perfect (100%) coverage on the test datasets for 14 benchmark domains. Yet such empirical evidence is limited to a set of example tasks (the test dataset). Do the generalized plans actually solve *all* tasks in the domains?

Stein et al. (2026) showed through a manual analysis that this is indeed the case (similar manual analyses were done by Drexler, Seipp, and Geffner (2022) and Francès, Bonet, and Geffner (2021)). But how to lead this kind of proof automatically? *Given the PDDL description of a domain, as well as some specification of the set of domain instances (tasks in the domain) that a generalized plan should solve, how to automatically obtain a generalized plan that provably solves all these instances?* We refer to this challenge as the problem of computing *complete* generalized plans.

Earlier work on this challenge identified a restricted class of (so-called one-dimensional) planning problems where complete generalized plans can be found automatically using logical reasoning techniques (Hu and Levesque 2010). Another strand of work has addressed the complementary challenge of, given a generalized plan, or as part of the computation of the generalized plan, determining a set of instances which the plan is guaranteed to solve (e.g. Srivastava, Immerman, and Zilberstein 2011; Bonet et al. 2019); the key concept in the most recent work on this are so-called sound abstractions, which underlie the generalized plan and guarantee faithfulness to a concrete set of instances. Recent work has shown how to automatically prove such abstractions sound, and therewith solve the challenge of proving a

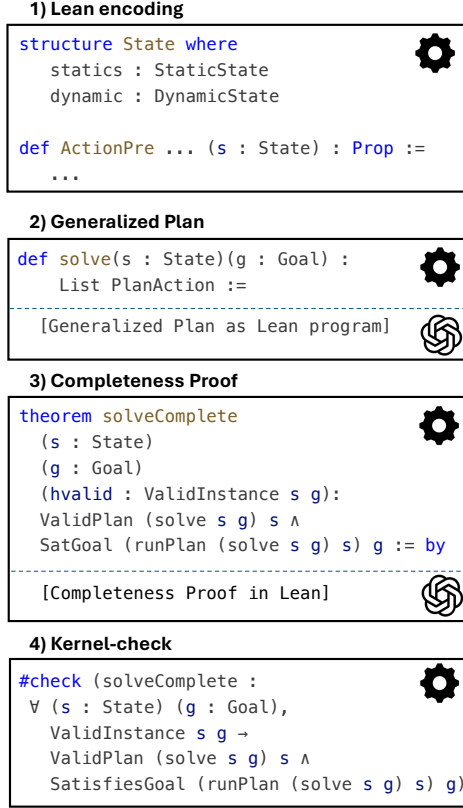


Figure 2: Illustrations of the main steps of our approach for obtaining generalized plans in Lean and proving their completeness.

given generalized plan complete (Cui, Kuang, and Liu 2023). The empirical success with this method is, however, limited to very simple domains, such as clearing a single block in Blocksworld, the IPC Gripper domain, and Logistics with a single vehicle.

Here we achieve a major step forward in empirical success by leveraging the power of LLMs to not only generate the generalized plans, but also the completeness proofs in a suitable formalization. Specifically, we prove completeness of LLM-generated generalized plans against the semantics of the actual PDDL domain using Lean (de Moura et al. 2015), a functional programming language and theorem prover. We generate the generalized plans directly in Lean, and automatically prove their completeness relative to *domain constraints* formalizing the set of instances.

We automatically generate Lean code from the PDDL enforcing the domain semantics. Following Silver et al. (2024); Stein et al. (2026), we let the LLM generate the generalized plan as a program that takes a problem instance as input and outputs a complete plan. We then prompt the LLM to write a proof for the following theorem: *Let $\mathbf{GP}$ be the generated generalized plan. If I is a problem instance satisfying the domain constraints, then $\mathbf{GP}(I)$ returns a plan where 1) all actions are sequentially applicable and 2) the resulting state satisfies the goal.* A valid proof for this theorem shows that

the generalized plan $\mathbf{GP}$ is complete.¹ Apart from the PDDL itself, our approach requires as input only a specification of the domain constraints, as well as a specification of state invariants. Everything else is completely automated, including the generation and debugging of $\mathbf{GP}$ as per Stein et al. (2026), and the proof that $\mathbf{GP}$ is complete. Figure 2 illustrates our approach.

We empirically evaluate our approach on 13 commonly used benchmark domains (a subset of the domains addressed by Stein et al. (2026)). Using GPT-5.6-Sol as the LLM, for 12 of these domains we obtain a generalized plan and a valid proof, showing automatically that the generalized plan is complete. On the remaining domain, we obtain a generalized plan with perfect coverage on the test set. This is a massive advancement of the state of the art in automatic generalized-plan completeness proofs.

Background

PDDL planning. In our work we focus on planning domains and problems defined in the Planning Domain Definition Language (PDDL) (McDermott 2000; Haslum et al. 2019). In PDDL, a planning task is specified by domain $\mathcal{D}$ and a problem instance I of the domain. A domain is a pair $\mathcal{D} = \langle \mathcal{P}_{\mathcal{D}}, \mathcal{A}_{\mathcal{D}} \rangle$ where $\mathcal{P}_{\mathcal{D}}$ defines the set of predicates for describing states and $\mathcal{A}_{\mathcal{D}}$ defines the set of action schemas. Each predicate $P \in \mathcal{P}_{\mathcal{D}}$ takes a finite number of arguments, i.e. has an arity of $k \geq 0$. Each action schema $A \in \mathcal{A}_{\mathcal{D}}$ is a triplet $\langle \text{Arg}(A), \text{Pre}(A), \text{Eff}(A) \rangle$ where $\text{Arg}(A)$ are the arguments of the action, $\text{Pre}(A)$ defines its preconditions and $\text{Eff}(A)$ its effects.

We focus on domains that can be expressed by a subset of PDDL that is restricted to conjunctive conditions with negation. Hence, all preconditions and effects are conjunctions of literals from $\mathcal{P}$ where all predicate arguments are also arguments of the action. We refer to the positive preconditions and effects as Pre^+ and Eff^+ , and to the negative preconditions and effects as Pre^- and Eff^- respectively. We require $\text{Eff}^+(A) \cap \text{Eff}^-(A) = \emptyset$.

A problem instance I is a triplet $\langle O, S_0, G \rangle$ where O is a finite set of objects, potentially with types, S_0 defines the initial state and G the goal conditions. Both S_0 and G are defined in terms of ground atoms, i.e. predicates of the domain where the arguments are replaced by objects $o \in O$, $p(o_1, \dots, o_k)$. The initial state forms a set of atoms and is interpreted under the closed-world assumption, which means that all atoms that are not listed as part of the initial state are false. In contrast, a goal is a conjunction of literals, $G = G^+ \cup G^-$, and is interpreted under the open-world assumption, i.e. the final truth value of all atoms that are not part of the goal specification is irrelevant. A state s satisfies a goal G iff $G^+ \subseteq s \wedge G^- \cap s = \emptyset$. We write $s \models G$ if s satisfies G .

Figure 3a shows an excerpt from the PDDL Spanner domain (3a) and Figure 3b shows an example of a problem instance.

¹In our benchmarks, we only consider solvable problem instances. If the domain constraints do not guarantee solvability, then there exists no proof of the theorem.

```
(:types
  location locatable - object
  man nut spanner - locatable)

(:predicates
  (at ?m - locatable ?l - location)
  (carrying ?m - man ?s - spanner)
  (useable ?s - spanner)
  (link ?l1 - location ?l2 - location)
  (tightened ?n - nut)
  (loose ?n - nut))

(:action walk
  :parameters (?s - location ?e - location ?m - man)
  :precondition (and (at ?m ?s)
                     (link ?s ?e))
  :effect (and (not (at ?m ?s)) (at ?m ?e)))
```

(a) Excerpt from the domain definition.

```
(:objects bob - man
  spanner1 spanner2 - spanner
  nut1 nut2 - nut
  location1 location2 shed gate - location)

(:init
  (at spanner1 location2) (useable spanner1)
  (at spanner2 location1) (useable spanner2)
  (loose nut1) (at nut1 gate)
  (loose nut2) (at nut2 gate)
  (at bob shed) (link shed location1)
  (link location1 location2) (link location2 gate))

(:goal
  (and (tightened nut1) (tightened nut2)))
```

(b) Example problem instance.

Figure 3: PDDL examples from the Spanner domain.

An action $a(o_1, \dots, o_k)$ is an instance of an action schema A grounded by objects from O . An action a is applicable in a state s iff all its preconditions are satisfied in s , i.e. $\text{Pre}^+(a) \subseteq s \wedge \text{Pre}^-(a) \cap s = \emptyset$. Applying an action a in a state s yields a successor state s' where $s' = (s \setminus \text{Eff}^-(a)) \cup \text{Eff}^+(a)$. We define the successor state function $\gamma : S \times A \rightarrow S$ where S denotes the set of all states as follows: If an action a is applicable in a state s then $\gamma(s, a) = s' = (s \setminus \text{Eff}^-(a)) \cup \text{Eff}^+(a)$. If a is not applicable then $\gamma(s, a)$ is undefined. We extend γ to sequences of actions $[a_1, \dots, a_n]$ by iterated application: $\gamma(s, []) = s$ and $\gamma(s, [a_1, \dots, a_n]) = \gamma(\gamma(s, a_1), [a_2, \dots, a_n])$, which is defined only if each a_i is applicable in the state resulting from applying $[a_1, \dots, a_{i-1}]$ to s . We say a plan π is applicable from a state s if $\gamma(s, \pi)$ is defined.

A plan $\pi = [a_1, \dots, a_n]$ is a solution for a planning task $\langle \mathcal{D}, I \rangle$ with $I = \langle O, S_0, G \rangle$ if and only if $\gamma(S_0, \pi)$ is defined and $\gamma(S_0, \pi)$ satisfies G .

Domains can be typed, i.e. define a set of objects types and a hierarchy on them. A typed domain can be converted into an equivalent untyped domain by converting types into

unary predicates.

Not all possible initial states and goals form a valid problem instance of a domain. The set of valid instances contains all instances that satisfy the constraints of the domain that are only implicitly defined by the respective problem generators. For example, the problem generator for the Spanner domain generates only instances where all nuts are loose in the initial state and need to be tightened in the end. Hence, an instance I belongs to $\mathcal{D}$ if it satisfies each constraint C from the set of initial and goal constraints $\mathcal{C}$. Furthermore, for our work an instance needs to be solvable in order to be considered valid.

Generalized planning. Research on generalized planning comprises a variety of different representations for generalized plans and different approaches to obtain them. These include policies based on features and abstractions that are hand-crafted (Bonet and Geffner 2018) or learned (Bonet, Francès, and Geffner 2019), policies for qualitative numerical problems (Srivastava et al. 2011), finite-state controllers that augment a policy with an internal memory (Bonet, Palacios, and Geffner 2009) and directly synthesized programs with control flow structures such as loops and conditionals (e.g. Jiménez, Segovia-Aguas, and Jonsson 2019; Srivastava, Immerman, and Zilberstein 2011).

More recently, approaches involving neural models have been proposed, including learning general policies with graph neural networks (Ståhlberg, Bonet, and Geffner 2022, 2025) and generating programmatic generalized plans using pretrained LLMs. Silver et al. (2024) and Stein et al. (2026) use LLMs to come up with strategies for solving all planning tasks from a specific domain and to implement these strategies in the form of Python functions that take a Python representation of a planning problem as input and return a sequence of actions. Their approaches include an automated debugging loop for the Python programs and Stein et al. (2026) also include an automated debugging step of the strategies. Here, we consider generalized plans in the form of executable programs that take a problem instance as input and return a plan.

Completeness of generalized plans. Completeness guarantees for generalized plans have been approached in different ways. One line of work restricts the representation of generalized plans itself. For this restricted class of so-called one-dimensional planning problems, complete generalized plans can be found automatically using logical reasoning constraints (Hu and Levesque 2010). A complementary line of work starts from a generalized plan and determines the characterization of the set of instances that the generalized plan solves (Bonet et al. 2019), or computes the characterization of the set of covered instances jointly with the generalized plan (Srivastava, Immerman, and Zilberstein 2011). The key concept underlying the most recent work in this direction is that of a sound abstraction: whenever the abstract generalized plan reaches the abstract goal, the resulting concrete plan is guaranteed to solve every concrete instance that is covered by the abstraction. Cui, Kuang, and Liu (2023) show how to automatically prove the soundness of such abstractions, and hence automatically prove that the generalized plan is complete. However, this has only been shown for very simple

domains (e.g. IPC Gripper) and restricted set-ups (Logistics with one vehicle) so far. Another approach for establishing completeness is manual analysis, which can be applied to a larger variety of domains but at the cost of manual work by experts. Stein et al. (2026), Drexler, Seipp, and Geffner (2022) and Francès, Bonet, and Geffner (2021) manually verified that their generated or hand-crafted generalized plans solve all instances of a domain.

Lean. Lean (de Moura and Ullrich 2021) is a dependently typed functional programming language which enables the implementation of a generalized plan as an executable, type-checked function directly in Lean. Furthermore, Lean is also an interactive theorem prover. So, it is used to write ordinary programs as well as to state and prove mathematical and program-correctness theorems about them. It is based on a dependent type theory where types may depend on values. Lean is built on the Curry-Howard correspondence, thus propositions are represented as types and a proof of a proposition is a term of that type. Stating a theorem therefore amounts to writing down a type, while proving it amounts to constructing a term of that type. A Lean representation is thus a sequence of *definitions* introducing functions and datatypes, *statements* expressing propositions and *proofs* constructing terms that inhabit the types of their propositions. Type-checking is what constitutes verification in Lean: if the term type-checks, the statement counts as proven since a proof of a proposition is by constructing a term of the corresponding type. Constructing such terms explicitly is impractical, so Lean offers a *tactic mode* to provide such proofs as a sequence of transformation steps, such as rewriting with an equation or performing induction. Lean executes this sequence to construct the explicit proof term. This term, along with definitions and previously proven lemmas it depends on, is provided to Lean’s *kernel*, a small trusted program that type-checks it.

Lean Representations and Correctness

Our objective is to obtain domain-specific generalized plans for PDDL planning domains and prove their completeness in Lean for any PDDL planning domain. This requires a conversion from PDDL to Lean that faithfully preserves the semantics of the planning task. In this section, we describe our Lean representation and the conversion.

We use the subscript “L” to indicate that a concept is represented in Lean and no subscript for PDDL, e.g. I refers to PDDL problem instance and I_L to a Lean problem instance. We write $\llbracket \cdot \rrbracket$ for the conversion operator that maps PDDL concepts to their Lean counterparts as defined in this section. $\llbracket I \rrbracket$ therefore refers to the specific problem instance obtained by applying the conversion to the PDDL instance I . We use the inversion of the operator, i.e., $\llbracket \cdot \rrbracket^{-1}$, to refer to the conversion from Lean concepts back to their original PDDL counterpart.

Representing States and Instances

States. Our Lean representation of a PDDL state is split into two parts: a **static state** and a **dynamic state** as shown in Listing 1. The static and dynamic parts together comprise

everything that is required for capturing all parts of a PDDL state: the set of objects, and a boolean-valued function for every predicate of the domain.

We represent objects as natural numbers $i \in \mathbb{N}$, i.e. our conversion maps each PDDL object $o \in O$ to a unique $i \in \mathbb{N}$ by enumeration, hence $\llbracket \cdot \rrbracket$ is bijective on O . Each PDDL predicate $P \in \mathcal{P}_D$ of arity k is represented as a boolean curried function $P_L : \mathbb{N}^k \rightarrow \text{Bool}$ that takes k objects as input and returns a truth value. Since the predicate functions are total functions relative to the set of objects, this matches the closed-world interpretation of PDDL states. All predicate functions for which the truth assignments cannot be changed by any action are part of the static state part together with the set of objects. For example, the predicate `(link ?l1 - location ?l2 - location)` becomes the function `link_p :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool}$` , and is part of the static state part because none of the action effects affect the connections between locations. All other predicate functions are part of the dynamic part. Following the standard approach, we compile away PDDL types by turning each type into a unary predicate, so types are handled like any other predicate.

Given the object mappings, a PDDL state s is converted into a Lean state s_L by defining the truth assignments of the predicate functions such that the following state correspondence relation holds uniquely:

Definition 1 (State Correspondence). Let s be a PDDL state, s_L a Lean state. s and s_L correspond to each other, $s \sim s_L$, if for every predicate P and tuple of objects $(o_1, \dots, o_k)$, $p(o_1, \dots, o_k) \in s$ iff $\llbracket p(o_1, \dots, o_k) \rrbracket = P_L(i_1, \dots, i_k)$ returns true, where $i_j = \llbracket o_j \rrbracket$.

Instances. Encoding a PDDL goal condition follows a similar principle, but differs in two key aspects. First, predicates in the goal are defined as partial functions in order to reflect the open-world assumption, as shown in Listing 1, where `Option Bool : none` means that the atom is unconstrained, while `some b` requires the state to assign it the truth value `b`. Second, the goal is defined only in terms of the predicate functions for which truth assignments can change, i.e. the dynamic part because if the static goal conditions match the initial state then they are trivially true for any reachable state. In contrast, if there is a contradiction between the static part of the initial state and the goal then the problem instance is not solvable at all, and hence not considered by us. We refer to the goal function for a predicate $P \in \mathcal{P}_D$ as PG_L .

A PDDL goal is converted into a Lean goal analogously to a state while preserving the open-world interpretation of goals: $\llbracket G \rrbracket$ is created such that for every predicate P of arity k and tuple of objects, $PG_L(i_1, \dots, i_k) = \text{true}$ iff $p(o_1, \dots, o_k) \in G^+$ and $PG_L(i_1, \dots, i_k) = \text{false}$ iff $p(o_1, \dots, o_k) \in G^-$ and is undefined otherwise. For example, if the goal specifies that `nut nut1` needs to be tightened but does not specify the location of the man `bob`, and $\llbracket \text{nut1} \rrbracket = 3$ and $\llbracket \text{bob} \rrbracket = 4$ then `tightened_pG(3)` returns true and `at_pG(4, o)` returns none for any $o \in O$ (see Figure 5 in Appendix for a full example of a converted problem instance).

The goal satisfaction relation. $\models_L$, is defined in Lean as a function `SatisfiesGoal : State  $\rightarrow$  Goal  $\rightarrow$  Prop`.

```

structure State where
  statics : StaticState
  dynamic : DynamicState

structure StaticState where
  objects : List N
  link_p : N → N → Bool
  location_t : N → Bool
  locatable_t : N → Bool
  ...

structure DynamicState where
  at_p : N → N → Bool
  carrying_p : N → N → Bool
  useable_p : N → Bool
  ...

structure Goal where
  dynamic : GoalDynamic

structure GoalDynamic where
  at_p : N → N → Option Bool
  carrying_p : N → N → Option Bool
  useable_p : N → Option Bool
  ...

```

Listing 1: Excerpt of the Lean state and goal representations of the PDDL Spanner domain (Figure 3a).

$\text{SatisfiesGoal } s \ g$ holds iff for every goal predicate and every argument, the corresponding predicate of the Lean state s agrees with the value that g requires wherever g requires one. We define the result as a `Prop` rather than a `Bool` because goal satisfaction appears in our completeness theorem (Theorem 2), and a theorem statement must be a proposition. Listing 2 shows an excerpt for the Spanner domain.

A complete PDDL and Lean problem instance then correspond to each other if the following holds:

Definition 2 (Instance correspondence). A PDDL instance $I = \langle O, S_0, G \rangle$ and a Lean instance $I_L = \langle [I], [S_0], [G] \rangle$ correspond to each other, i.e. $I \sim I_L$, if $S_0 \sim [S_0]$ and for any state s : $s \models G$ iff $[s] \models_L [G]$

By construction, our conversion ensures that for each PDDL instance I , $I \sim [I]$.

Representing Actions and Transitions

Actions. Actions are encoded as an inductive type `PlanAction` with one constructor per PDDL action schema (Listing 3). We use an inductive type here rather than a structure as we did for states, because a structure in Lean has a single constructor, whereas actions form a choice among several schemas. Lean requires that any function over `PlanAction`, such as the transition function, handles every constructor. Each constructor fixes the arity of its schema, so that ill-formed ground actions are rejected by Lean’s type checker. A Lean ground action is simply a constructor applied to all its arguments, e.g. `walk(1, 2, 3)`. Since all ground

```

def SatisfiesGoal (s : State) (g : Goal) :
  Prop :=
  (∀ var_m var_l,
    match g.dynamic.at_p var_m var_l with
    | none => True
    | some b => s.dynamic.at_p var_m var_l
      = b) ∧
  (∀ var_n,
    match g.dynamic.tightened_p var_n with
    | none => True
    | some b => s.dynamic.tightened_p var_n
      = b) ∧
  ...

```

Listing 2: Example of the Lean function checking for goal satisfaction.

```

inductive PlanAction where
  | walk (var_s : N) (var_e : N) (var_m : N)
  | pickup_spanner (var_l : N) (var_s : N) (var_m : N)
  | tighten_nut (var_l : N) (var_s : N) (var_m : N) (var_n : N)

```

Listing 3: Example of the action constructors for the action schemas from the Spanner domain.

actions thus share the type `PlanAction`, a **Lean plan** π_L is of type `List PlanAction`.

Preconditions and effects for each action schema are represented as a pair of Lean functions. Like `SatisfiesGoal`, the precondition function for an action schema A takes the arguments of the encoded action and Lean state s_L to return a proposition that the grounded action is applicable in s_L . This function is defined as a conjunction where the conjuncts correspond to the Lean conversions of the literals in $\text{Pre}(A)$. Since state predicates are Boolean-valued, each conjunct is an equation requiring the corresponding predicate function to return `true` for a positive literal, or `false` for a negative literal. Listing 4 shows an example for the preconditions of the “walk” action schema from the Spanner domain.

The effect function for an action schema A takes the same input as the precondition function and returns the updated state s'_L , in which the predicate function of each add effect in $\text{Eff}(A)$ returns `true` at the affected arguments, each delete effect returns `false`, and all other truth assignments are unchanged. Listing 4 shows this for the “walk” schema, where only `at_p` is affected. The Lean syntax `{s with dynamic := ...}` builds a copy of s in which only the named field differs, and the nested `s.dynamic with ...` analogously replaces only `at_p`; every other predicate function is carried over unchanged. The new `at_p` returns `true` for the man’s new location `var_e`, `false` for the previous location `var_s`, and the previous truth value for all other argument pairs.

```

def walkPre (var_s : N) (var_e : N) (var_m : N) (s : State) : Prop :=
  s.dynamic.at_p var_m var_s = true ∧
  s.statics.link_p var_s var_e = true ∧
  s.statics.location_t var_s = true ∧
  s.statics.location_t var_e = true ∧
  s.statics.man_t var_m = true

def walkEff (var_s : N) (var_e : N) (var_m : N) (s : State) : State :=
  {s with dynamic := {
    s.dynamic with
    at_p :=
      fun var_m' var_e' =>
        if var_m' = var_m ∧ var_e' = var_e
        then
          true
        else if var_m' = var_m ∧ var_e' = var_s
        then
          false
        else
          s.dynamic.at_p var_m' var_e' }}

```

Listing 4: Example of the Lean representation of the preconditions and effects of the action schema ‘walk’.

Transitions. By construction, for any ground action a of an action schema A and any state s the following holds: the Lean precondition function returns true on $\llbracket a \rrbracket$ and $\llbracket s \rrbracket$ iff a is applicable in s and if so, then the effect function applied to $\llbracket a \rrbracket$ and $\llbracket s \rrbracket$ returns a state s'_L that corresponds to the PDDL successor state $s' = \gamma(s, a)$.

The functions defined so far are specific to a single action schema, whereas the transitions of a planning task are defined for arbitrary ground actions. We therefore combine them into `actionPre` and `actionApply` (Listing 5), which take an arbitrary `PlanAction` and a state as input. Lean accepts these definitions only if they match exhaustively over every constructor of `PlanAction`, so all action schemas of the domain are covered, each branch calling the precondition or effect function of its corresponding schema. Together, `actionPre` and `actionApply` form the Lean counterpart γ_L to the PDDL successor state function γ , with $\gamma_L(s_L, a_L) := \text{actionApply}(a_L, s_L)$ if `actionPre`(a_L, s_L) is true and undefined otherwise. Unlike γ , `actionApply` is a total function by itself, and reproduces the partial definition of γ in combination with `actionPre`. Figure 4 (left) illustrates the following lemma:

Lemma 1 (Action Transitions Commute). *For any state s and ground action a , $\gamma(s, a)$ is defined iff $\gamma_L(\llbracket s \rrbracket, \llbracket a \rrbracket)$ is defined. If $\gamma(s, a) = s'$ and $\gamma_L(\llbracket s \rrbracket, \llbracket a \rrbracket) = s'_L$ then $s'_L = \llbracket s' \rrbracket$.*

From Single Steps to Plan Correctness

We lift applicability and state updates from single actions to sequences by defining two recursive functions, `ValidPlan` and `runPlan` (Listing 6), the former stating the proposition under which a plan is applicable and the second computing the state it produces. The empty plan is always appli-

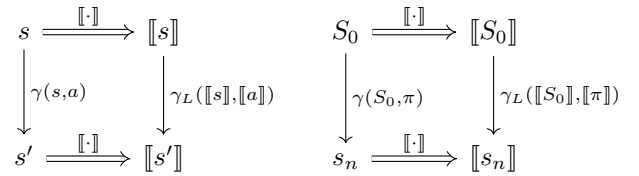
```

def actionPre : PlanAction → State → Prop
| .walk var_s var_e var_m, s => walkPre var_s var_e var_m s
| .pickup_spanner var_l var_s var_m, s => pickup_spannerPre var_l var_s var_m s
| .tighten_nut var_l var_s var_m var_n, s => tighten_nutPre var_l var_s var_m var_n s

def actionApply : PlanAction → State → State
| .walk var_s var_e var_m, s => walk var_s var_e var_m s
| .pickup_spanner var_l var_s var_m, s => pickup_spanner var_l var_s var_m s
| .tighten_nut var_l var_s var_m var_n, s => tighten_nut var_l var_s var_m var_n s

```

Listing 5: Example of the Lean applicability and state update functions for the Spanner domain.



(a) Single-action transition.

(b) Full-plan transition.

Figure 4: Conversion $\llbracket \cdot \rrbracket$ commutes with each applicable ground action when applying this single action (left). Iterating the square over a full plan shows that $\llbracket \cdot \rrbracket$ also commutes with applying a sequence of actions (right).

cable and leaves the state unchanged. For a non-empty list, `ValidPlan` requires the first action to be applicable in the current state and the remaining actions to form a valid plan from the state that results from applying it, while `runPlan` applies the first action and continues on the remaining list. Notice that `runPlan` applies an action regardless of whether its preconditions hold and applicability is captured separately by `ValidPlan`. Checking for the correctness of a plan hence requires both: a plan π_L is a solution for a problem instance with the initial state S_{0L} and the goal G_L iff `ValidPlan`(π_L, S_{0L}) is true and `runPlan`(π_L, S_{0L}) satisfies G_L (see `correctSolution`, Listing 6).

Since initial states are states and since a single action corresponds to a plan of length 1, Lemma 1 applies to the initial state of a problem instance and extends to plans by induction over plan length, as illustrated in Figure 4 (right).

By construction, our PDDL to Lean conversion ensures that a PDDL problem instance I and its converted Lean counterpart $\llbracket I \rrbracket$ correspond. Therefore, a PDDL plan and its Lean counterpart agree on applicability and goal satisfaction:

Lemma 2 (Plan Correctness Transfers to Conversion). *Let*

```

def ValidPlan : List PlanAction → State →
  Prop
| [], _ => True
| a :: as, s => actionPre a s ∧ ValidPlan
  as (actionApply a s)

def runPlan : List PlanAction → State →
  State
| [], s => s
| a :: as, s => runPlan as (actionApply a
  s)

def correctSolution (s : State) (g : Goal)
  (plan : List PlanAction) : Prop
ValidPlan plan s ∧
SatisfiesGoal (runPlan plan s) g

```

Listing 6: Lean functions for checking plan applicability and correctness and for executing a plan.

$I = \langle O, S_0, G \rangle$ be an instance of $\mathcal{D}$, and let $\pi = [a_1, \dots, a_n]$ be a sequence of actions. Then π is a correct plan solving I iff $\llbracket \pi \rrbracket$ is a correct plan solving the corresponding Lean planning problem $\llbracket I \rrbracket$.

Since $\llbracket \cdot \rrbracket$ is bijective on objects and each PDDL action schema corresponds to exactly one `PlanAction` constructor, $\llbracket \cdot \rrbracket$ is bijective on ground action sequences and hence every Lean plan π_L has a unique PDDL counterpart $\llbracket \pi_L \rrbracket^{-1}$.

Theorem 1 (Soundness of Lean Plan Verification). *Let $I = \langle O, S_0, G \rangle$ be an instance of $\mathcal{D}$. If we obtain a sequence of actions, $\pi_L : \text{List PlanAction}$, and applying `correctSolution` to $\llbracket S_0 \rrbracket$, $\llbracket G \rrbracket$ and π_L returns true then converting π_L from Lean to PDDL gives a plan $\pi = \llbracket \pi_L \rrbracket^{-1}$ such that π is guaranteed to be a solution of the original PDDL problem instance I .*

Theorem 1 follows directly from Lemma 2: `correctSolution`($\llbracket S_0 \rrbracket$, $\llbracket G \rrbracket$, π_L) is by definition the Lean statement that π_L is a correct plan solving $\llbracket I \rrbracket$ and the bidirectionality of Lemma 2 gives that $\llbracket \pi_L \rrbracket^{-1}$ is a correct plan solving I .

Completeness of Generalized Plans

We now lift the correctness from individual plans to the completeness of generalized plans. We define a generalized plan as a (domain-specific) Lean function `solve` that takes a Lean problem instance, separated into the initial state and the goal, as input and returns a plan in the form of a list of `PlanAction` for the input problem instance. Listing 7 shows the signature of a generalized plan.

Not every problem instance expressible in Lean needs to be handled by the `solve` function. As described in the Background, the actual set of problem instances of a PDDL domain is restricted by a set of constraints C on valid initial states and goals. We represent these constraints in Lean as two predicates `ValidInit` : $\text{State} \rightarrow \text{Prop}$ and `ValidGoal` : $\text{State} \rightarrow \text{Goal} \rightarrow \text{Prop}$. Completeness of a generalized plan `solve` is then defined as follows:

```

def solve (s : State) (g : Goal) : List
  PlanAction :=
  -- Implementation of the generalized plan
  GP

```

Listing 7: Signature of the generalized plan `solve`, which takes a state and a goal and returns a plan.

```

solveComplete : ∀ (s : State) (g : Goal),
  ValidInit s → ValidGoal s g →
  ValidPlan (solve s g) s ∧
  SatisfiesGoal (runPlan (solve s g) s) g

```

Listing 8: Type of the completeness theorem statement for the generalized plan `solve`: for any valid initial state and goal, `solve` returns an applicable plan that reaches the goal.

Definition 3 (Generalized Plan Completeness). Let $\mathcal{D}$ be a domain. A Lean function `solve` with the signature of Listing 7 is a complete generalized plan for $\mathcal{D}$ if, for every initial state s_L and every goal G_L that satisfy `ValidInit`(s_L) and `ValidGoal`(s_L , G_L), the plan returned by `solve` $s_L G_L$ is applicable from s_L and the resulting state satisfies G_L .

Definition 3 guarantees the correctness of the generated plans for an infinite number of problem instances because there are infinitely many pairs of valid initial states and goals. It is expressed as a Lean theorem of the type shown in Listing 8 which expresses exactly the universal implication statement. Listing 9 shows the actual Lean declaration of the theorem which can be completed by a proof candidate that can then be checked by Lean’s kernel. A Lean proof of `solveComplete` (Listing 9) for a specific candidate `solve` function establishes exactly the completeness definition (Def. 3) for that candidate if accepted by Lean’s kernel:

Theorem 2 (Completeness GP Lean). *Let $\mathcal{D}$ be a domain and let GP_L be a candidate generalized plan for $\mathcal{D}$, i.e. a Lean function with the signature shown in Listing 7. If there exists a proof for `solveComplete` (Def. 3) with `solve` := GP_L , that is accepted by Lean’s kernel, then GP_L is a complete generalized plan for $\mathcal{D}$: for every problem*

```

theorem solveComplete
  (s : State)
  (g : Goal)
  (hinit : ValidInit s)
  (hgoal : ValidGoal s g) :
  ValidPlan (solve s g) s ∧
  SatisfiesGoal (runPlan (solve s g) s) g
  := by
  -- Proof term of the theorem

```

Listing 9: Lean declaration of the completeness theorem statement in Listing 8.

instance I_L satisfying the domain constraints, applying GP_L to I_L returns a plan that is applicable from the initial state and reaches the goal.

Theorem 2 itself requires no separate proof beyond Lean’s own soundness guarantee that whenever the kernel accepts a proof term of a proposition then the proposition holds. Theorem 2 is, however, about the correctness of a generalized plan GP_L with respect to Lean problem instances and not the original PDDL. We extend the correctness theorem in order to carry over to PDDL based on Lemma 2 and Theorem 1. From Lemma 2 and Theorem 1 we get that, given our conversion $\llbracket \cdot \rrbracket$, every PDDL instance I of a domain has a corresponding Lean instance, and every Lean plan that solves the converted problem instance has a PDDL counterpart that solves the original PDDL instance. Therefore, correctness of solve in Lean transfers to a correctness guarantee for the original PDDL.

Corollary 1 (Completeness of GP for PDDL Domain). *Let GP_L be a complete generalized plan in Lean for a domain $\mathcal{D}$ (Theorem 2), and let $\llbracket \cdot \rrbracket$ be our PDDL to Lean conversion. Then for every $I = \langle O, S_0, G \rangle \in \mathcal{I}_{\mathcal{D}}$, $GP_L(\llbracket I \rrbracket)$ returns a plan π_L such that $\pi_L = \llbracket \pi \rrbracket$ for some plan π that solves I .*

Validity Constraints

The completeness theorem in Listing 9 is proven relative to `ValidInit` and `ValidGoal`: these two Lean predicates specify which problem instances a generalized plan is required to solve by defining the constraints on valid problem instances, which we call validity constraints. They hence define what “the complete domain” means given a generalized plan. There are two main reasons for restricting the possible input space. First, we only focus on solvable instances and require the validity constraints to exclude unsolvable ones, since otherwise there are instances for which `ValidInit` and `ValidGoal` holds but the conclusion of the completeness theorem could never hold for them. Therefore, no proof would exist.

Second, even after excluding unsolvable instances, it is not feasible to create a generalized plan that would cover every possible combination of ground atoms: the number of syntactically correct initial states and goals is much larger than the number of possible instances that a real problem generator can produce, and that were actually targeted by the creator of the domain. Many combinations are not even structurally sensible: for example, it should not be possible that a location is linked to a spanner but only the other way round, and no object should be a spanner and a nut at the same time. If a PDDL domain is originally typed, these constraints can be automatically extracted based on the defined type hierarchy and the types assigned to the predicate parameters. `ValidTypeHierarchy` and `ValidLinkParam` in Listing 10 show examples of one of these constraints. For untyped domains, these kinds of requirements need to be inferred from the implementation of the problem generator or from a representable set of problem instances.

The additional constraints on valid initial states and goals can also only be inferred based on data or the generators because none of these assumptions is part of the PDDL def-

```
def AtLeastAsManySpannersAsNuts (s :
  StaticState) : Prop :=
  (s.objects.filter (fun x => s.nut_t x)).
    length ≤ (s.objects.filter (fun x =>
      s.spanner_t x)).length

def ValidTypeHierarchy (s : StaticState) :
  Prop :=
  (∀ x, s.nut_t x = true → s.spanner_t x =
    false) ∧
  ...

def ValidLinkParam (s : StaticState) : Prop
:=
  ∀ var_l1 var_l2, s.link_p var_l1 var_l2 =
    true → s.location_t var_l1 = true ∧
    s.location_t var_l2 = true

def LinkAcyclic (s : StaticState) : Prop :=
  ∀ l, ¬ Relation.TransGen (fun a b => s.
    link_p a b = true) l l

def InitNutsLoose (s : State) : Prop :=
  ∀ n, s.statics.nut_t n = true → s.dynamic
    .loose_p n = true

def GoalAllNutsTightened (initial : State)
  (g : Goal) : Prop :=
  ∀ n, initial.statics.nut_t n = true → g.
    dynamic.tightened_p n = some true

def GoalIgnoreAt (initial : State) (g :
  Goal) : Prop :=
  ∀ o l, g.dynamic.at_p o l = none
```

Listing 10: Examples of constraints on valid instances of the Spanner domain.

inition. For example, the problem generator for the Spanner domain generates only instances for which all nuts are loose initially but must be tightened in the end, as specified by the `InitNutsLoose` and `GoalAllNutsTightened` constraints respectively. The `GoalIgnoreAt` constraint illustrates how the open-world interpretation of goals is operationalized in the constraints.

Lean’s logic is the calculus of inductive constructions (Pfenning and Paulin-Mohring 1990), used for defining relations by induction and quantifying over predicates. This allows directly expressing properties such as reachability and hence the absence of cycles of any length. Thus constraints such as the location graph in Spanner forming a linear sequence are straightforward as we can define the transitive closure of `link_p` and state that no location is reachable from itself (`LinkAcyclic`).

The validity of an initial state (`ValidInit`) is then defined as a conjunction of the initial state constraints and validity of a state in general, which itself is again a conjunction of the constraints on the dynamic state part and the constraints on the static state part. Similarly, goal validity of a state (`ValidGoal`) is defined as a conjunction of the goal

constraints.

Although there has been work on automatically inferring constraints on valid problem instances, no existing approach provides constraints with the guarantees that are required for proving correctness of generalized plans. We therefore use a hybrid approach here: all constraints that can be derived based on the PDDL, i.e. from type hierarchies and typed predicates, are generated automatically and all other constraints need to be defined manually by a human.

Given that there is no actual set $\mathcal{C}$ of constraints on a PDDL domain available, it is the responsibility of the person who wants to obtain a generalized plan for a domain to actually define what exactly “the complete domain” means for them. The completeness of a generalized plan is therefore proven against the specification provided by a human.

Generalized Planning in Lean with LLMs

The previous section described the theoretical foundation of our approach and the prerequisites for representing generalized plans for PDDL planning tasks in Lean and proving their completeness. In this section, we now describe how we obtain the generalized plans and completeness proofs. Given a PDDL domain and the specifications of the validity constraints, our pipeline 1) automatically generates its Lean representation, 2) lets an LLM generate a candidate generalized plan `solve`, and 3) uses again an LLM to generate the proof that `solve` satisfies the correctness theorem from Listing 9. Only the specification of the constraints on the input problems requires manual work. Every other step, including potential debugging of the LLM-generated Lean code, is fully automated.

Getting Lean representations.

In order to obtain the Lean formalizations, we implemented a domain-independent Python converter that takes as input the PDDL domain, and optionally a PDDL problem, and outputs the converted Lean representation, i.e. the state, action and instance representations described above (see Listings 1, 3-5). Our implementation for processing the PDDL covers a subset of PDDL that is restricted to conjunctive conditions with negation, no action costs, and can be applied to typed and untyped domains². The converter is purely symbolic and deterministic³, and applies the conversion described in the previous section in the same structured way to any domain of the covered PDDL fragment. Therefore, all claims and guarantees that depend on the specific conversion mechanism described before hold for the Lean representation of any domain obtained from our converter.

Generating Provably Complete Generalized Plans

Generating the generalized plan in Lean. Generating generalized plans with LLMs consists of two main steps:

²In order to extend the approach to a larger PDDL fragment only the PDDL processing part needs to be adapted.

³The only exception with respect to the determinism concerns the order of iterations, e.g. in which the actions are converted and objects are enumerated. None of the iteration orderings, however, has any effect on the validity of our results.

1) deriving the strategy for solving a planning domain, and 2) implementing that strategy in a programming language. Previous work has shown that LLMs can perform well on both tasks when combined with automatic debugging, and in particular when generating strategies in the form of pseudocode (Stein et al. 2026). Their resulting generalized plans are automatically validated on a finite set of test tasks, but checking their completeness required manual evaluation. Our work addresses this gap by implementing and proving generalized plans in Lean, hence automating the evaluation of the completeness. As our focus is on this step rather than on finding strategies, we start from the pseudocode strategies of Stein et al. (2026), check them and revise them if necessary. We provide an LLM with the revised strategies as well as the complete Lean representation of the target domain and have it implement the `solve` function (Listing 7) such that `correctSolution` (Listing 6) holds. This code is debugged in two steps. Statically, Lean’s kernel must accept the code. Dynamically, the candidate generalized plan is run on a set of debugging tasks and the plans it returns are converted to PDDL and validated with VAL (Howey, Long, and Fox 2004). As VAL is independent of our encoding, it is used to ensure the PDDL domain is converted correctly. If either step fails, the LLM receives feedback about the specific error and is prompted to revise its generated code. If the generated candidate generalized plan returns correct solutions for all debugging tasks within a defined number of iterations, the process moves on to the proof step.

Generating the completeness proof. Once a generalized plan has been generated, we ask the LLM to prove that it is correct, i.e., to state and prove the correctness theorem shown in Listing 9. As input, the LLM receives (i) the complete Lean code from the previous step, consisting of the Lean representation and the LLM-generated generalized plan, (ii) lemmas stating that actions preserve state validity, described in the next section, and (iii) helper lemmas describing which parts of a state an action changes and which it leaves unchanged. These helper lemmas are generated automatically from the PDDL by a symbolic, domain-independent generator (see Listing 13 and 14 in Appendix).

We again use an automatic debugging procedure to let the LLM revise its generated proof in case the generated proof is rejected. If Lean reports any error, or the proof fails one of the checks against cheating described below, the error message is passed back to the LLM, together with a prompt asking it to regenerate the complete proof.

Depending on the specific planning domain and the concrete implementation of the generalized plan, completeness can be hard to prove and the simple, basic approach described may not work. In particular, we observe that in these cases the LLM explicitly replies that proving completeness is too complex. To address this we propose a second, iterative approach inspired by Ospanov, Farnia, and Yousefzadeh (2025), which, as our experiments show, works better for those complex cases. The LLM is given the same information as in the basic approach but is instructed to provide a proof sketch that states all lemmas it requires. Notably, this approach allows the LLM to decide which lemmas it proves immediately in its

first response and which are marked unproven for now. The remaining lemmas are then proven in subsequent iterations. The completeness proof is only accepted once it depends on no unproven lemmas.

To debug the proof sketch, we split it into its individual declarations, i.e. definitions, lemmas and theorems. These declarations are added to the Lean representation one at a time, checking the code after each addition. When a declaration is rejected, the LLM revises the declaration based on the code accepted so far and the error message. Thus, errors are repaired locally without discarding the parts Lean has already accepted in contrast to the basic approach where the complete proof needs to be regenerated. The process ends when all declarations are accepted or the maximum number of iterations is reached. During the debugging, the LLM is explicitly allowed to generate more declarations than the one that failed to account for declarations missing in the initial sketch. They are then inserted into the proof sketch at the same place as the declaration that is currently being debugged.

Preventing LLM cheating. LLMs generating Lean code are known to find workarounds that make the code compile without establishing the intended result (Wang et al. 2026; Paglieri et al. 2026), and recent work analyzing Lean benchmarks has found related evaluation-time loopholes that can affect the reliability of evaluation results (Ammanamanchi, Bhat, and Biderman 2026). There are three ways in which this can happen in our setting. First, the LLM may change `correctSolution` either by directly editing the statement itself, e.g. adding hypotheses that make the statement easier to prove, or by altering the semantics, e.g. by redefining notation or introducing new definitions (Paglieri et al. 2026). Second, the proof may silently rely on unjustified axioms which do not appear in the theorem’s type. Third, a proof may be accepted while bypassing Lean’s kernel checks due to metaprogramming or bugs in incremental compilation within Lean.

We address these with four checks. We reject code that alters how subsequent code is parsed or elaborated, such as custom notation or macros. We further verify that the final code contains a theorem exactly as written in Listing 8. Together, these rule out the first form of cheating. For the second form of cheating, we check that the proof does not depend on axioms beyond the standard ones.⁴ Finally, for efficiency, our iterative debugging loop is performed incrementally in REPL.⁵ As a sanity check, the complete code is rebuilt from scratch as a self-contained module and rechecked by Lean’s standalone checker, thus ruling out the third form of cheating as well.

Obtaining Validity Constraints

Getting validity constraints. As the domain constraints on valid instances need to be specified manually, we define them for the 13 domains that we used for our experiments based on the problem generators, example problems from datasets,

⁴<https://lean-lang.org/doc/reference/latest/Axioms/#standard-axioms>

⁵<https://github.com/leanprover-community/repl>

```
def NutTightenedLooseExhaustive (s : State)
  : Prop :=
  ∀ n, s.statics.nut_t n = true
  → s.dynamic.tightened_p n = true ∨
    s.dynamic.loose_p n = true
```

Listing 11: Examples of a state invariant constraint of the Spanner domain.

potential domain descriptions, and the PDDL domain definition. In addition to the types of constraints described above, we specify a set of state invariants: properties that hold in the initial state and remain true in any state reachable from it, such as `NutTightenedLooseExhaustive` shown in Listing 11 which specifies that each nut is always either tightened or loose. Every invariant of this kind is a consequence of the initial state constraints together with the action schemas. In practice, however, we provide these invariants explicitly as part of the valid state specification because doing so removes the burden of re-deriving them from scratch from the LLM. At the same time, adding them to the constraints does not add a substantial additional amount of work. For example, without invariants, the initial state constraints need to include both a constraint that all nuts are loose (`InitNutsLoose`) and one specifying that no nut is tightened. When adding `NutTightenedLooseExhaustive` as an invariant constraint, the constraint about all nuts being not tightened becomes redundant.

Checking validity constraints. While it is not possible to automatically check the correctness of the constraints, we conduct two automatic sanity checks to catch some potential mistakes. First, we check that all constraints are jointly satisfiable because otherwise the correctness statement could be trivially proven by a vacuous proof. We therefore take one planning task from an existing dataset of the target domain and convert it into its Lean representation. We then automatically generate one theorem per constraint that proves that this constraint holds for the converted planning task. Listing 12 shows an example of a constraint theorem statement. For obtaining the actual proof body we employ a hybrid approach: first a number of general proof tactics, e.g. `simp all`, are inserted one after the other as the proof body and checked by Lean’s kernel checker. If none of the generic tactics leads to an accepted proof, an LLM is prompted to either complete the proof or state that the theorem does not hold. If all theorems are accepted by the Lean checker, then all constraints hold for the selected example task and are therefore jointly satisfiable. Otherwise, the constraints need to be manually revised.

Additionally, we conduct a sanity check of the specified invariants. In particular, we define a `ValidState` as a conjunction over all constraints that apply to arbitrary states (i.e. not only initial states). We then automatically generate one lemma per action schema for proving that applying the specific action in a valid state yields again a valid state. Then, an LLM is prompted to generate the proof or state that no such proof exists in case the lemma does not hold. If the

```

theorem witness_init_InitNutsLoose :
  InitNutsLoose problemState := by
-- TODO:

lemma walk_preserves_wf
  (var_s var_e var_m)
  (s : State)
  (hwf : ValidState s)
  (hpre : walkPre var_s var_e var_m s) :
  ValidState (walk var_s var_e var_m s)
:= by
-- TODO:

```

Listing 12: Examples of theorems for the sanity checks of the validity constraints.

latter happens for at least one of the action lemmas, then the constraints need to be revised.

Experiments

Pipeline and Set-up

We test our approach using GPT-5.6-Sol with temperature set to 1 and reasoning set to high. We select 13 of the 17 domains on which also Stein et al. (2026) ran their experiments. We exclude Gripper because of its similarity to Grippers, Beluga because LLMs struggle to find generalized plans there (Stein et al. 2026) and Minigrid and Visitall as the specification of solvability is not straightforward⁶. For all the domains that we test our approach on except Rovers, Stein et al. (2026) manually proved that their generalized plans were complete. For each domain, we select six small problem instances from existing datasets as debugging tasks (see Table 3 in Appendix for sources of datasets). For the satisfiability check of the constraints, we use the smallest of the selected debugging tasks. In case the smallest problem instance is still considerably larger than the smallest possible problem instances, we manually adapt it and reduce its size. This additional modification is added because the size of the instance affects the complexity of the proofs for the constraint satisfaction. And since the objective of these proofs is to show that there is at least one instance for which the constraints are satisfiable a smaller instance simplifies this task without affecting its validity.

Constraints

For each of our defined validity constraints, our approach first tries to prove that the selected problem instance satisfies the constraint using standard tactics, as described above. In this way, between 69% (Blocksworld) and 91% (Satellite) of the constraints can already be proven without an LLM. For each of the remaining constraints, the LLM is then prompted to provide the proofs and can refine its reply up to five times if necessary based on the error feedback. In our experiments, at most two rounds of debugging per constraint were required.

⁶Stein et al. (2026) used a version of Visitall that allows missing cells in the grid

For the sanity checks of the state invariants, we also include an automatic debugging loop such that the LLM has up to five tries for generating the proofs that each action returns again a valid state. Here, the LLM is prompted to generate, and debug, the proofs for all actions together instead of each action individually. The maximum number of debugging iterations in our experiments was three, with an average of 2.4 across the 13 domains.

For each case where the LLM replied that no proof could be generated because the problem instance does not actually satisfy the target constraints or because an action results in a state that does not satisfy the invariants we re-checked the defined constraints again and found that the LLM correctly identified a problem with the constraints. We then fixed the constraints and re-ran the sanity checks until all proofs were successful.

GP and proof.

The approach for generating the generalized plan (GP) described above is applied for obtaining the GPs for all 13 domains. We set the maximum number of debugging iterations to four. Table 1 presents the results. For all 13 domains, the LLM successfully generated generalized plans that were accepted by Lean and returned plans that were accepted by VAL for all six debugging tasks. The number of interactions (N interactions) refers to the number of input-output pairs of the LLM generation process for the generalized plan (GP) and the proof (Prf.), and includes both the initial versions generated by the LLM as well as the debugging iterations. For 10 of the domains, the first generated version of the generalized plan already solved the debugging tasks, and for the other three domains a single debugging iteration was enough. Given that we made sure to already provide the LLM a correct strategy for solving the domain, a low number of debugging iterations is expected here.

As described above, we consider two different approaches for generating the correctness proofs: a basic approach where the complete proof is generated and debugged as a whole and an iterative approach where the LLM first generates a sketch of the proof and then debugs the individual declarations (e.g., lemmas, definitions, theorems, etc.).

We first tested the basic approach for all domains with the maximum iterations for debugging set to six. For 7 of the 13 domains, the LLM successfully completed the correctness statement from Listing 9 with a proof that was accepted by Lean’s kernel. The number of generated proof versions (i.e. initial + debugging iterations) and the time required for obtaining the generalized plans and proofs for these 7 domains are shown in the upper part of Table 1.

For the other six domains, the LLM was not able to generate the correctness proof using the basic set-up. In particular, for all of these domains the LLM explicitly stated in the first response that providing a complete proof at once is not possible due to its complexity and the number of required additional lemmas. We therefore ran the iterative approach for those six domains. As the debugging is applied at the level of individual declarations here, we set the debugging limit such that the debugging terminates if the same declaration (as identified by the name of e.g. the lemma) is debugged

six times in a row without success. Using the iterative approach, we successfully obtain proofs accepted by Lean’s kernel for all domains except Transport. The lower part of Table 1 shows the maximum number of iterations required for the same declaration (Decl.) as well as the total number of interactions (Prf.) for obtaining the complete correct proof.

Note, however, that the lack of a proof for Transport does not mean that the generalized plan does not solve all instances of the domain, but that the LLM was not capable of generating one. In order to still get an idea of how well the generalized plan performs, we tested it on the easy and medium test splits of the IPC Learning track 2023 (Taitler et al. 2024) on which it achieved perfect coverage.

Domain	N interactions			Time		
	GP	Prf.	Decl.	GP	Prf.	total
Basic proof generation						
Delivery	1	3	–	121	671	792
Ferry	2	3	–	104	586	690
Grippers	2	3	–	119	661	780
Heavy	1	3	–	36	495	531
Hiking	1	2	–	113	582	694
Logistics	1	3	–	157	1025	1182
Satellite	1	4	–	224	1383	1607
Iterative proof generation						
Blocks.	2	24	3	315	1505	1819
Goldminer	1	36	2	290	2524	2814
Miconic	1	11	2	204	1302	1506
Rovers	1	45	3	307	2914	3221
Spanner	1	36	4	214	2735	2950
Transport	1	–	–	239	–	–

Table 1: Number of LLM interactions and run time of the pipeline (in seconds) for generating the generalized plan (GP) and its correctness proof (Prf.). “N interactions” counts the number of interactions, i.e. input-output pairs, with the LLM, including the initial generation and all debugging iterations. The upper part shows the domains for which the basic approach succeeded, the lower part shows the results using the iterative approach on the other domains. For the iterative approach, “Decl.” reports the maximum number of debugging iterations required for a single declaration.

Table 1 also shows how long it takes to obtain the generalized plans and the correctness proofs. The required time varies a lot between the different domains, ranging from 9 minutes for Heavy to almost one hour (54 minutes) for Rovers. Generating the generalized plans takes significantly less time even if the number of iterations is only slightly higher. As shown in Table 2, the Lean code for the proofs consists also of significantly more individual declarations, both with respect to the number of declarations the LLM generates and the number of declarations already provided to the LLM in the prompt. The number of declarations provided to the LLM is necessarily larger for the proof generation because it includes the complete Lean code (provided and LLM-generated) for the generalized plan.

Domain	N decl. GP		N decl. Prf.	
	given	LLM	given	LLM
Basic proof generation				
Delivery	56	3	92	15
Ferry	45	7	83	7
Grippers	51	9	102	23
Heavy	47	5	76	9
Hiking	45	5	65	16
Logistics	61	16	133	46
Satellite	56	10	116	17
Iterative proof generation				
Blocks.	52	10	122	42
Goldminer	78	16	194	71
Miconic	55	7	103	27
Rovers	120	17	293	66
Spanner	60	9	104	46

Table 2: Number of Lean code declarations (i.e., lemmas, theorems, and definitions) involved in generating the generalized plan (GP) and its correctness proof (Prf.). “Given” reports the number of declarations provided to the LLM as part of the prompt (e.g., the domain formalization and, for proof generation, the complete GP code); “LLM” reports the number of individual declarations generated by the LLM itself.

Related Work

In our work we formalize classical planning problems in Lean for getting completeness guarantees on generalized plans. Previous work has worked on formalizing classical planning domains for the purpose of verification. One line of work formalizes the semantics of a PDDL fragment directly inside an interactive theorem prover: Abdulaziz and Lam-mich (2018) formalize STRIPS-like PDDL in Isabelle/HOL and derive a formally verified plan validator, and follow-up work extends this to a verified compositional planning algorithm (Abdulaziz, Gretton, and Norrish 2019) and a formally verified SAT-based planner (Abdulaziz and Kurz 2023). Another line of work lets planners emit independently checkable certificates alongside their output, without formalizing the planner itself: Eriksson, Röger, and Helmert (2017) introduce certificates of unsolvability that a planner-independent validator can verify, and Mugdan, Christen, and Eriksson (2023) extend this approach of certifying algorithms to certificates of plan optimality. Like our work, both lines of work target machine-checked guarantees in the context of classical planning but verify a fixed planning algorithm’s output, e.g. single plans, (un)solvability claims, rather than verifying correctness across the entire set of domain instances.

Software verification. Verifying the correctness of synthesized programs has long been studied in software verification and program synthesis. Classical work established that a verifier can be used not only to check a synthesized program in the end, but to guide synthesis itself, iteratively refining candidate programs against counterexamples until a verified implementation is found (Solar-Lezama et al. 2006). At a larger scale, formally verified compilers such as Com-

pCert demonstrate that machine-checked correctness proofs, i.e. checked by a proof assistant, can be obtained for realistic, complex software artifacts rather than only small synthesized fragments (Leroy 2009).

More recently, this combination of synthesis and verification has been extended to use LLMs as the source of the candidate programs. Bhatia et al. (2024) use an LLM to jointly generate a program and a proof of its functional equivalence to a source program, translating both into an automated theorem prover to verify correctness for all program states rather than testing on a finite test set. Aggarwal, Parno, and Welleck (2025) similarly address formally verified code generation by iteratively translating and refining candidate programs against feedback from a formal verifier, and Chen et al. (2025) fine-tune models on synthesized proof data for proof generation and debugging in Rust.

In all these approaches, an external, trusted verifier provides the correctness guarantee for an LLM-generated program, e.g. a transpiled function (Bhatia et al. 2024) or a Verus-annotated Rust function (Aggarwal, Parno, and Welleck 2025), but the guarantees are only checked once against one specification. We adopt the same overall pattern here, but for programs that must remain correct across every valid instance of a planning domain, not a single specific input-output specification.

LLMs for theorem proving. First et al. (2023) introduce an approach to automatically generate and repair complete proofs using LLMs based on feedback from the Isabelle proof assistant to iteratively debug incorrect generated proofs. They fine-tune models and evaluate them on a benchmark of general mathematical theorems rather than software or planning related theorems. Song, Yang, and Anandkumar (2024) integrate LLM-based tactic suggestions, proof search and premise selection directly into the Lean workflow, focusing on interactively developing proofs for math. Most closely related to our proof-debugging approach is the work by Ospanov, Farnia, and Yousefzadeh (2025): they isolate failing sub-lemmas in an LLM-generated proof sketch and repair them individually, using a hybrid approach that combines automated solvers and LLM calls, instead of debugging the whole proof at once. Their focus is on competition-style mathematical benchmarks.

Learning constraints. Most work on learning or obtaining constraints for a planning domain focuses on invariants, i.e. properties that hold across all states reachable from any initial state and that therefore follow directly from the constraints of initial states and action semantics. Methods for learning invariants derive these properties, such as, for example, predicate disjunctions and monotonicity conditions, that provably hold for any reachable state (e.g. Rintanen 2000; Lin 2004; Mukherji and Schubert 2005; Helmert 2009) but cannot derive constraints that come from the problem generators, e.g., that all spanners are initially usable and all nuts loose. Shleyfman and Karpas (2018) and Grundke, Röger, and Helmert (2024) propose to extend PDDL domain definitions by the types of constraints that usually come from the problem generators. They introduce formalisms for specifying these constraints along the PDDL but still require that

constraints are manually specified. Recent work by Gestrin, Fichte, and Seipp (2026) formalizes the problem of learning exactly these kinds of constraints from a set of positive example instances and proposes an abstract framework for learning these constraints that also covers existing approaches for learning invariants. Gestrin, Fichte, and Seipp (2026) show that their framework comes with the guarantee of probably approximately correct convergence with respect to a specific candidate constraint language. However, this guarantee only holds in the limit of infinitely many examples and is therefore not strong enough to be used for our correctness proofs.

Conclusion

We presented an approach for automatically obtaining generalized plans together with machine-checked completeness proofs by using LLMs to generate both of them directly in Lean. Our approach only requires the PDDL definitions of the target domain as well as the domain constraints and invariants, and fully automates the conversion between PDDL and Lean, and the generation, debugging and verification of the generalized plans and completeness proofs. Across 13 benchmark domains, our approach produced generalized plans with perfect coverage on the test sets, and for 12 of these domains also produced a kernel-checked proof that the plan is complete, i.e. not just empirically successful, but provably correct for all problem instances satisfying the domain constraints.

Acknowledgments

The work was partially funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under the project number 232722074 – SFB 1102. It was also funded in part by the Deutsche Forschungsgemeinschaft – GRK 2853/1 “Neuroexplicit Models of Language, Vision, and Action” - project number 471607914. We thank Jan Reineke, Dan Fišer and Nicola Müller for insightful discussions.

References

- Abdulaziz, M.; Gretton, C.; and Norrish, M. 2019. A Verified Compositional Algorithm for AI Planning. In Harrison, J.; O’Leary, J.; and Tolmach, A., eds., *10th International Conference on Interactive Theorem Proving (ITP 2019)*, volume 141 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 4:1–4:19. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik. ISBN 978-3-95977-122-1.
- Abdulaziz, M.; and Kurz, F. 2023. Formally Verified SAT-Based AI Planning. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI 2023)*, 14665–14673. AAAI Press.
- Abdulaziz, M.; and Lammich, P. 2018. A Formally Verified Validator for Classical Planning Problems and Solutions. In *2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI)*, 474–479.
- Aggarwal, P.; Parno, B.; and Welleck, S. 2025. AlphaVerus: bootstrapping formally verified code generation through self-improving translation and tree refinement. In *Proceedings of*

- the 42nd International Conference on Machine Learning, ICML'25. JMLR.org.
- Ammanamanchi, P. S.; Bhat, S.; and Biderman, S. 2026. Faults in Our Formal Benchmarking: Dataset Defects and Evaluation Failures in Lean Theorem Proving. arXiv:2606.29493.
- Bhatia, S.; Qiu, J.; Hasabnis, N.; Seshia, S.; and Cheung, A. 2024. Verified Code Transpilation with LLMs. In Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J.; and Zhang, C., eds., *Advances in Neural Information Processing Systems*, volume 37, 41394–41424. Curran Associates, Inc.
- Bonet, B.; Francès, G.; and Geffner, H. 2019. Learning Features and Abstract Actions for Computing Generalized Plans. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI 2019)*, 2703–2710. AAAI Press.
- Bonet, B.; Fuentetaja, R.; E-Martín, Y.; and Borrajo, D. 2019. Guarantees for Sound Abstractions for Generalized Planning. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI 2019)*, 1566–1573. IJCAI.
- Bonet, B.; and Geffner, H. 2018. Features, Projections, and Representation Change for Generalized Planning. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI 2018)*, 4667–4673. IJCAI.
- Bonet, B.; Palacios, H.; and Geffner, H. 2009. Automatic Derivation of Memoryless Policies and Finite-state Controllers Using Classical Planners. In *Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling (ICAPS 2009)*, 34–41.
- Chen, T.; Lu, S.; Lu, S.; Gong, Y.; Yang, C.; Li, X.; Misu, M. R. H.; Yu, H.; Duan, N.; CHENG, P.; Yang, F.; Lahiri, S.; Xie, T.; and Zhou, L. 2025. Automated Proof Generation for Rust Code via Self-Evolution. In Yue, Y.; Garg, A.; Peng, N.; Sha, F.; and Yu, R., eds., *International Conference on Learning Representations*, volume 2025, 71891–71918.
- Cui, Z.; Kuang, W.; and Liu, Y. 2023. Automatic Verification for Soundness of Bounded QNP Abstractions for Generalized Planning. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI 2023)*, 3149–3157. IJCAI.
- de Moura, L.; Kong, S.; Avigad, J.; van Doorn, F.; and von Raumer, J. 2015. The Lean Theorem Prover (System Description). In Felty, A. P.; and Middeldorp, A., eds., *Automated Deduction - CADE-25*, 378–388. Cham: Springer International Publishing.
- de Moura, L.; and Ullrich, S. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction - CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, 625–635. Berlin, Heidelberg: Springer-Verlag. ISBN 978-3-030-79875-8.
- Drexler, D.; Seipp, J.; and Geffner, H. 2022. Learning Sketches for Decomposing Planning Problems into Subproblems of Bounded Width. In *Proceedings of the Thirty-Second International Conference on Automated Planning and Scheduling (ICAPS 2022)*, 62–70. AAAI Press.
- Eriksson, S.; Röger, G.; and Helmert, M. 2017. Unsolvability Certificates for Classical Planning. In *Proceedings of the Twenty-Seventh International Conference on Automated Planning and Scheduling (ICAPS 2017)*, 88–97. AAAI Press.
- First, E.; Rabe, M. N.; Ringer, T.; and Brun, Y. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023*, 1229–1241. Association for Computing Machinery. ISBN 9798400703270.
- Francès, G.; Bonet, B.; and Geffner, H. 2021. Learning General Planning Policies from Small Examples Without Supervision. In *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI 2021)*, 11801–11808. AAAI Press.
- Gestrin, E.; Fichte, J.; and Seipp, J. 2026. Automatically Uncovering Intended Domain Constraints in Automated Planning. Workshop on Knowledge Engineering for Planning and Scheduling (KEPS).
- Grundke, C.; Röger, G.; and Helmert, M. 2024. Formal Representations of Classical Planning Domains. In *Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS)*, 239–248.
- Haslum, P.; Lipovetzky, N.; Magazzeni, D.; and Muise, C. 2019. *An Introduction to the Planning Domain Definition Language*, volume 13 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Morgan & Claypool Publishers.
- Helmert, M. 2009. Concise finite-domain representations for PDDL planning tasks. *Artificial Intelligence*, 173(5): 503–535. Advances in Automated Plan Generation.
- Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *Proceedings of the 16th International Conference on Tools with Artificial Intelligence (ICTAI 2004)*, 294–301. IEEE.
- Hu, Y.; and Levesque, H. J. 2010. A Correctness Result for Reasoning about One-Dimensional Planning Problems. In *Proceedings of the Twelfth International Conference on Principles of Knowledge Representation and Reasoning (KR 2010)*, 362–371. AAAI Press.
- Jiménez, S.; Segovia-Aguas, J.; and Jonsson, A. 2019. A review of generalized planning. *The Knowledge Engineering Review*, 34: e5.
- Leroy, X. 2009. Formal Verification of a Realistic Compiler. *Communications of the ACM*, 52(7): 107–115.
- Lin, F. 2004. Discovering State Invariants. *KR*, 4: 536–544.
- McDermott, D. M. 2000. The 1998 AI planning systems competition. *AI magazine*, 21(2): 35–55.
- Mugdan, E.; Christen, R.; and Eriksson, S. 2023. Optimality Certificates for Classical Planning. In *Proceedings of the Thirty-Third International Conference on Automated Planning and Scheduling (ICAPS 2023)*, 286–294. AAAI Press.

Mukherji, P.; and Schubert, L. K. 2005. Discovering Planning Invariants as Anomalies in State Descriptions. In *Proceedings of the Fifteenth International Conference on Automated Planning and Scheduling (ICAPS 2005)*, 223–230.

Ospanov, A.; Farnia, F.; and Yousefzadeh, R. 2025. APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Paglieri, D.; Cross, L.; Genewein, T.; Leibo, J. Z.; Tomasev, N.; and Vezhnevets, A. S. 2026. A Case Study on Emergent Cheating and Whistleblowing in Autonomous Research Swarms. arXiv:2609.04170.

Pfenning, F.; and Paulin-Mohring, C. 1990. Inductively defined types in the Calculus of Constructions. In Main, M.; Melton, A.; Mislove, M.; and Schmidt, D., eds., *Mathematical Foundations of Programming Semantics*, 209–228. New York, NY: Springer-Verlag. ISBN 978-0-387-34808-7.

Rintanen, J. 2000. An Iterative Algorithm for Synthesizing Invariants. In *Proceedings of the Seventeenth National Conference on Artificial Intelligence (AAAI 2000)*, 806–811.

Seipp, J.; Torralba, Á.; and Hoffmann, J. 2022. PDDL Generators. <https://doi.org/10.5281/zenodo.6382173>.

Shleyfman, A.; and Karpas, E. 2018. Position Paper: Reasoning About Domains with PDDL. In *The 2018 AAAI Spring Symposium Series*, 582–586.

Silver, T.; Dan, S.; Srinivas, K.; Tenenbaum, J.; Pack Kaelbling, L.; and Katz, M. 2024. Generalized Planning in PDDL Domains with Pretrained Large Language Models. In Dy, J.; and Natarajan, S., eds., *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence (AAAI 2024)*, 20256–20264. AAAI Press.

Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS XII*, 404–415. New York, NY, USA: Association for Computing Machinery. ISBN 1595934510.

Song, P.; Yang, K.; and Anandkumar, A. 2024. Towards Large Language Models as Copilots for Theorem Proving in Lean. *CoRR*, abs/2404.12534.

Srivastava, S.; Immerman, N.; and Zilberstein, S. 2011. A new representation and associated algorithms for generalized planning. *Artificial Intelligence*, 175(2): 615–647.

Srivastava, S.; Zilberstein, S.; Immerman, N.; and Geffner, H. 2011. Qualitative Numeric Planning. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence (AAAI 2011)*, 1010–1016. AAAI Press.

Ståhlberg, S.; Bonet, B.; and Geffner, H. 2022. Learning general optimal policies with graph neural networks: Expressive power, transparency, and limits. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 32, 629–637.

Ståhlberg, S.; Bonet, B.; and Geffner, H. 2025. Learning More Expressive General Policies for Classical Planning Domains. In *Proceedings of the Thirty-Ninth AAAI Conference*

on Artificial Intelligence (AAAI 2025), 26697–26706. AAAI Press.

Stein, K.; Hodel, N.; Fišer, D.; Hoffmann, J.; Katz, M.; and Koller, A. 2026. Improved generalized planning with LLMs through strategy refinement and reflection. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 36, 677–686.

Taitler, A.; Alford, R.; Espasa, J.; Behnke, G.; Fišer, D.; Gimelfarb, M.; Pommerening, F.; Sanner, S.; Scala, E.; Schreiber, D.; Segovia-Aguas, J.; and Seipp, J. 2024. The 2023 International Planning Competition. *AI Magazine*, 45(2): 280–296.

Wang, J.; Zhang, J.; Guo, Q.; Guo, L.; Li, R.; Zhang, C.; Peng, C.; Wang, C.; Zhao, D.; Shi, J.; Wang, J.; Feng, L.; Shen, M.; Li, Q.; An, S.; Wang, S.; Shi, W.; Xi, X.; Li, X.; Cao, X.; Lu, Y.; Zhao, Y.; Chen, Z.; Lin, Z.; Wang, W.; Pei, P.; and Cai, X. 2026. LongCat-Flash-Prover: Advancing Native Formal Reasoning via Agentic Tool-Integrated Reinforcement Learning. arXiv:2603.21065.

Datasets

Table 3 lists the datasets from which the debugging problem instances and the example for the validity checks were taken, as well as the instance generators based on which the validity constraint specifications were created.

Additional Representation Examples

Figure 5 shows a side-by-side example of a PDDL problem instance with two spanners, two nuts and four locations (including shed and gate) (left part) and the corresponding Lean problem instance (right part).

Listing 13 shows examples of the additional, domain-specific lemmas about how states change (and do not change) that are generated symbolically based on the PDDL and provided as helper lemmas to the LLM. Listing 14 shows additional symbolically generated lemmas that the LLM is provided for an domain.

Domain	Source of debugging problems	Instance Generator
delivery		Silver et al. (2024)
ferry		IPC Learning track 2023 (Taitler et al. 2024)
heavy		Silver et al. (2024)
hiking		Silver et al. (2024)
miconic		Silver et al. (2024)
spanner		IPC Learning track 2023 (Taitler et al. 2024)
blocksworld		IPC Learning track 2023 (Taitler et al. 2024)
goldminer	Stein et al. (2026)	Seipp, Torralba, and Hoffmann (2022)
grippers	Stein et al. (2026)	Seipp, Torralba, and Hoffmann (2022)
logistics	Stein et al. (2026)	Seipp, Torralba, and Hoffmann (2022)
rovers		IPC Learning track 2023 (Taitler et al. 2024)
satellite		IPC Learning track 2023 (Taitler et al. 2024)
transport	Stein et al. (2026)	Seipp, Torralba, and Hoffmann (2022)

Table 3: The origin of the debugging tasks that we used for our experiments and the instance generators based on which we created the validity constraints.

<pre> (define (problem prob0) (:domain spanner) (:objects bob - man spanner1 - spanner spanner2 - spanner nut1 - nut nut2 - nut loc1 - location loc2 - location shed - location gate - location) (:init (at bob shed) (at spanner1 loc2) (useable spanner1) (at spanner2 loc1) (useable spanner2) (loose nut1) (at nut1 gate) (loose nut2) (at nut2 gate) (link shed loc1) (link loc1 loc2) (link loc2 gate)) (:goal (and (tightened nut1) (tightened nut2)))) </pre>	<pre> def problemStatics : StaticState := { objects := [0, 1, 2, 3, 4, 5, 6, 7, 8] link_p := fun x0 x1 => (x0 == 5 && x1 == 6) (x0 == 6 && x1 == 8) (x0 == 7 && x1 == 5), location_t := fun x0 => (x0 == 5) (x0 == 6) (x0 == 7) (x0 == 8), locatable_t := fun x0 => (x0 == 0) (x0 == 1) (x0 == 2) (x0 == 3) (x0 == 4), man_t := fun x0 => (x0 == 0), nut_t := fun x0 => (x0 == 3) (x0 == 4), spanner_t := fun x0 => (x0 == 1) (x0 == 2) } def problemDynamic : DynamicState := { at_p := fun x0 x1 => (x0 == 0 && x1 == 7) (x0 == 1 && x1 == 6) (x0 == 2 && x1 == 5) (x0 == 3 && x1 == 8) (x0 == 4 && x1 == 8), carrying_p := fun x0 x1 => false, useable_p := fun x0 => (x0 == 1) (x0 == 2), tightened_p := fun x0 => false, loose_p := fun x0 => (x0 == 3) (x0 == 4) } def goalDynamic : GoalDynamic := { at_p := fun x0 x1 => none, carrying_p := fun x0 x1 => none, useable_p := fun x0 => none, tightened_p := fun x0 => if x0 == 3 then some true else if x0 == 4 then some true else none, loose_p := fun x0 => none } </pre>
--	---

Figure 5: Example of a PDDL definition of a problem instance from the Spanner domain (left) and the corresponding, converted Lean problem instance (right).

```

-- walk does not change any predicate function from the static state part
lemma walk_statics (var_start var_end var_m : Obj) (s : State) :
  (walk var_start var_end var_m s).statics = s.statics := rfl

-- walk changes at_p for the man walking and the end location to true
lemma walk_at_p_eq1 (var_start var_end var_m : Obj) (s : State) :
  (walk var_start var_end var_m s).dynamic.at_p var_m var_end = true := by
  unfold walk
  simp

-- walk never changes at_p for locations other than the start and end location parameters
lemma walk_at_p_ne_var_1 (var_start var_end var_m : Obj) (s : State) {var_end' : Obj} (h1 :
  var_end' ≠ var_end) (h2 : var_end' ≠ var_start) :
  (walk var_start var_end var_m s).dynamic.at_p var_m var_end' = s.dynamic.at_p var_m
  var_end' := by
  unfold walk
  simp [h1, h2]

-- walk never changes carrying_p
lemma walk_carrying_p (var_start var_end var_m : Obj) (s : State) :
  (walk var_start var_end var_m s).dynamic.carrying_p = s.dynamic.carrying_p := rfl

-- Applying any plan does not change any predicate functions from the static state part
lemma runPlan_statics (plan : List PlanAction) (s : State) :
  (runPlan plan s).statics = s.statics := by
  induction plan generalizing s with
  | nil => simp [runPlan]
  | cons a as ih =>
    simp only [runPlan]
    rw [ih]
    cases a with
    | walk var_s var_e var_m => exact walk_statics var_s var_end var_m s
    | pickup_spanner var_l var_s var_m => exact pickup_spanner_statics var_l var_s var_m s
    | tighten_nut var_l var_s var_m var_n => exact tighten_nut_statics var_l var_s var_m
      var_n s

-- Applying an action in a valid state yields a valid successor state
lemma action_preserves_wf
  {a : PlanAction}
  {s : State}
  (hwf : WellFormed s)
  (hpre : actionPre a s) :
  WellFormed (actionApply a s) := by
  cases a with
  | walk var_start var_end var_m =>
    exact walk_preserves_wf var_start var_end var_m s hwf hpre
  | pickup_spanner var_l var_s var_m =>
    exact pickup_spanner_preserves_wf var_l var_s var_m s hwf hpre
  | tighten_nut var_l var_s var_m var_n =>
    exact tighten_nut_preserves_wf var_l var_s var_m var_n s hwf hpre

```

Listing 13: Examples of the additional, symbolically generated, lemmas provided the LLM for generating the correctness proof for the Spanner domain.

```

-- Applying a valid plan in a valid state yields a valid state
lemma validPlan_preserves_wf
  {plan : List PlanAction}
  {s : State}
  (hplan : ValidPlan plan s)
  (hwf : WellFormed s) :
  WellFormed (runPlan plan s) := by
induction plan generalizing s with
| nil =>
  simp [runPlan]
| cons a as ih =>
  simp only [ValidPlan] at hplan
  rcases hplan with ⟨ha, has⟩
  have hwf' :=
    action_preserves_wf (s := s) hwf ha
  exact ih has hwf'

-- Additional lemmas for combining sub plans
lemma runPlan_append (as bs : List PlanAction) (s : State) :
  runPlan (as ++ bs) s = runPlan bs (runPlan as s) := by
induction as generalizing s with
| nil => simp [runPlan]
| cons a as ih => simp [runPlan, ih]

lemma validPlan_append (as bs : List PlanAction) (s : State) :
  ValidPlan (as ++ bs) s ↔ ValidPlan as s ∧ ValidPlan bs (runPlan as s) := by
induction as generalizing s with
| nil => simp [ValidPlan, runPlan]
| cons a as ih => simp [ValidPlan, runPlan, ih, and_assoc]

```

Listing 14: The additional, symbolically generated, lemmas provided the LLM for generating the correctness proof for any domain.